

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей

Кафедра: Электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«КАЛЬКУЛЯТОР ЖИЗНИ»
Вариант 9

Студент:

А. Д. Бондарев

Преподаватель:

Д. В. Куприянова

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – освоить практические приёмы работы в интегрированной среде разработки IntelliJ IDEA при создании объектно-ориентированного Java-приложения с графическим интерфейсом, изучить процессы сборки, запуска и отладки программ, а также освоить проектирование приложений на основе архитектурного паттерна MVC с активной моделью.

В соответствии с вариантом 9 требуется разработать утилиту программу «Калькулятор жизни», которая по введённой дате рождения оценивает количество прожитых дней, время сна, число морганий, сердечных сокращений, объём перекачанной крови, количество выпитой воды и число случаев смеха с использованием усреднённых статистических коэффициентов. Для выполнения работы поставлены следующие задачи:

1. Создать проект Java в IntelliJ IDEA и настроить средства редактирования, сборки, запуска и отладки.
2. Создать главную форму с кнопкой «Ввести данные» и отдельное модальное окно для ввода даты рождения.
3. Обеспечить запоминание последней корректной даты и её восстановление при повторном вводе.
4. Реализовать многоуровневую проверку корректности даты и вывод понятных сообщений об ошибках.
5. Реализовать расчёт времени сна, количества морганий, сердечных сокращений, перекачанной крови, выпитой воды и смеха.
6. Разделить приложение на Model, View и Controller и реализовать активную модель через PropertyChangeSupport.
7. Проверить корректные и ошибочные сценарии работы программы и подготовить отчёт.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЕ

2.1 Интегрированная среда разработки и её компоненты

Современная интегрированная среда разработки представляет собой комплекс программных средств, объединённых в едином интерфейсе для повышения производительности труда программиста. В типичный состав IDE входят: текстовый редактор с подсветкой синтаксиса, компилятор или интерпретатор, система автоматической сборки проекта, отладчик с точками останова, встроенный терминал, модуль интеграции с системами контроля версий, а также средства навигации по коду и автоматического рефакторинга.

2.2 Редактирование и навигация в IntelliJ IDEA

Среда предоставляет широкий набор инструментов для ускоренной работы с исходным кодом. К основным возможностям редактора относятся: интеллектуальное автодополнение (Ctrl+Space), автоматическая вставка импортов, быстрое исправление ошибок через Alt+Enter, форматирование кода по стандарту (Ctrl+Alt+L), а также безопасное переименование идентификаторов (Shift+F6), при котором изменения автоматически применяются ко всем связанным файлам.

Для перемещения по проекту используются: переход к объявлению символа (Ctrl+B), поиск всех использований (Alt+F7), просмотр иерархии классов (Ctrl+N) и структурная панель проекта (Project). Результаты компиляции и работы программы отображаются в окнах Build и Run, а предупреждения анализатора – в окне Problems.

2.3 Сборка, запуск и отладка Java-приложения

Исходные тексты программы на языке Java преобразуются компилятором javac в байт-код, исполняемый виртуальной машиной JVM. В IntelliJ IDEA сборка проекта выполняется командой Build Project, а для запуска приложения создаётся конфигурация запуска, указывающая главный класс с методом main.

Отладка программы производится в специальном режиме Debug. Точка останова устанавливается щелчком мыши в левом поле редактора или сочетанием клавиш Ctrl+F8. При достижении точки останова выполнение приостанавливается, и в окне Debug отображаются стек вызовов, значения локальных переменных и полей объектов. Пошаговое выполнение осуществляется командами Step Over (перешагнуть), Step Into (войти в метод) и Step Out (выйти из метода). Для вычисления произвольных выражений в контексте текущего кадра используется Evaluate Expression.

2.4 Паттерн MVC

MVC (Model-View-Controller) – это архитектурный шаблон проектирования, согласно которому приложение разделяется на три слабосвязанных компонента. Model содержит данные и предметную логику, View отвечает за графический интерфейс и взаимодействие с пользователем, Controller обрабатывает действия пользователя и связывает представление с моделью. Такое разделение ответственности снижает связанность классов, упрощает тестирование бизнес-логики независимо от графического интерфейса и облегчает дальнейшее сопровождение и модификацию приложения.

2.5 Активная и пассивная модель

Различают два подхода к организации взаимодействия модели и представления.

При пассивной модели контроллер после изменения данных сам вызывает обновление представления.

При активной модели представление регистрируется в модели как слушатель, а модель самостоятельно уведомляет его об изменении состояния. В Java для этого удобно использовать `PropertyChangeSupport` и `PropertyChangeListener`.

2.6 Проверка корректности ввода данных

Для даты выполняется многоуровневая проверка: пустое поле, длина строки, разделители, допустимые символы, диапазоны дня и месяца, год, количество дней в месяце, високосный год, невозможность будущей даты и ограничение максимального возраста. При ошибке выводится сообщение, а модель не изменяется.

2.7 Сохранение и восстановление последних данных

Последняя корректная дата хранится в модели. Перед открытием окна ввода контроллер получает её и передаёт `InputDialog`. При наличии предыдущей даты она автоматически устанавливается в `JTextField`.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разработана программа «Калькулятор жизни». Дата рождения вводится вручную в формате ДД.ММ.ГГГГ без использования стандартного элемента выбора даты. После подтверждения программа определяет количество прожитых полных дней и рассчитывает статистические показатели.

В программе приняты коэффициенты: 8 часов сна в сутки, 15 морганий в минуту, 70 сердечных сокращений в минуту, 5 литров крови в минуту, 2 литра воды в сутки и 17 случаев смеха в сутки. Коэффициенты используются как учебные усреднённые значения.

Расчёт выполняется по формулам:

$$\begin{aligned}T_{\text{сна}} &= D \times 8, \\B &= D \times 24 \times 60 \times 15, \\H &= D \times 24 \times 60 \times 70, \\K &= D \times 24 \times 60 \times 5, \\W &= D \times 2, \\L &= D \times 17.\end{aligned}$$

где D – количество прожитых полных дней; Tсон – часы сна; B – моргания; H – сердечные сокращения; K – литры перекачанной крови; W – литры воды; L – случаи смеха.

3.2 Настройка среды и расширений для UML и SysML v2

Проект разработан на языке Java в IntelliJ IDEA. В качестве Project SDK рекомендуется использовать JDK 21.

Для UML может использоваться плагин PlantUML Integration. Диаграмма MVC подготовлена в файле src/mvc.puml. Для SysMLv2 в отчёте представлено текстовое структурное описание компонентов проекта и использовано совместимое средство моделирования, доступное в Settings – Plugins.

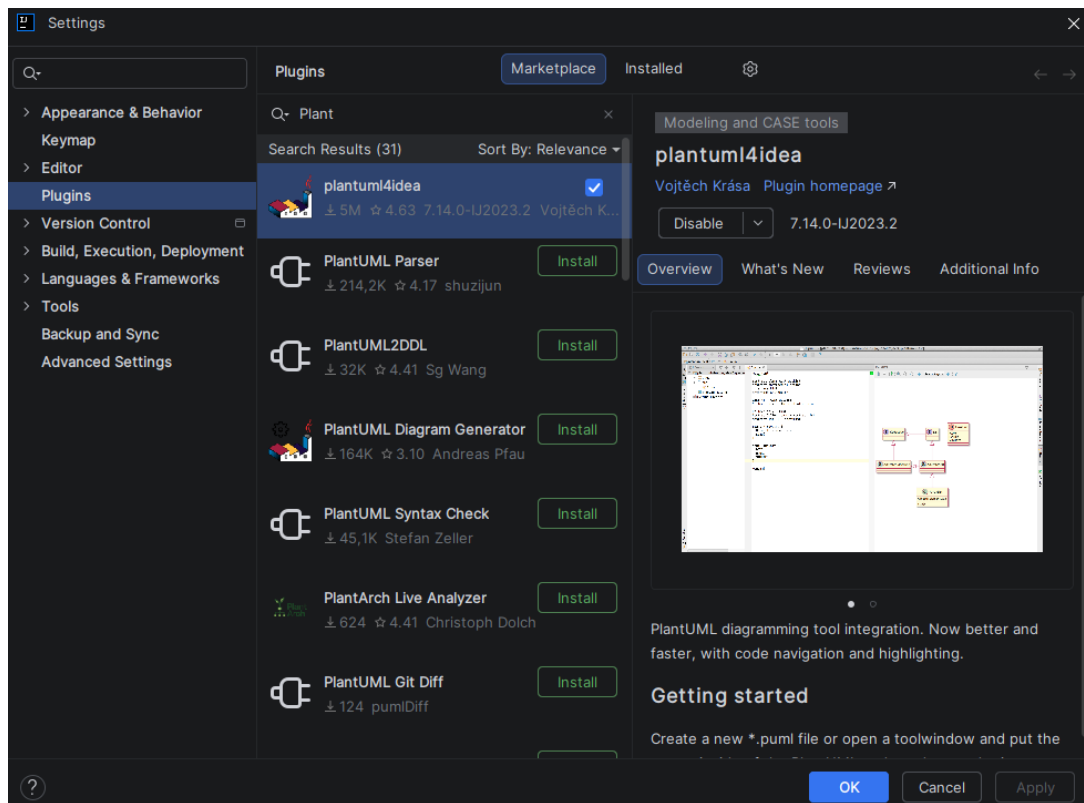


Рисунок 3.1 – Настройка проекта и средства моделирования IntelliJ IDEA

Структурное представление системы в нотации:

```
package LifeCalculator {
    part def LifeModel;
    part def MainView;
    part def InputDialog;
    part def LifeController;
    part application {
        part model : LifeModel;
        part view : MainView;
        part dialog : InputDialog;
        part controller : LifeController;
        connect view to controller;
        connect controller to model;
        connect model to view;
    }
}
```

3.3 Демонстрация отладчика

Работа отладчика может быть продемонстрирована в методе `setBirthDate()` класса `LifeModel` или `saveDate()` класса `InputDialog`. Точка останова позволяет проследить передачу даты, вычисление количества дней и рассылку `PropertyChangeEvent`.

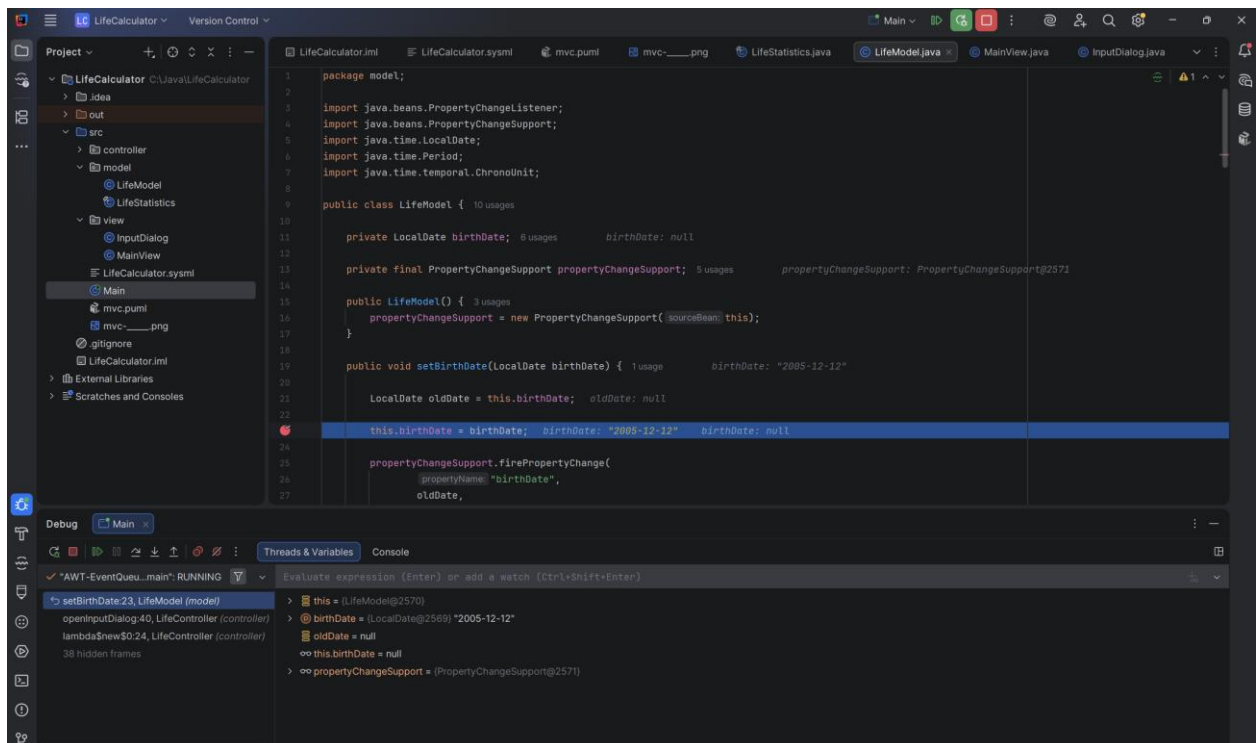


Рисунок 3.2 – Остановка программы в отладчике IntelliJ IDEA

После запуска программы в режиме Debug выполнение приостанавливается на точке останова. В окне Debug можно проследить значения birthDate, oldDate и промежуточных результатов. Команда Step Over позволяет пошагово пройти вычисления.

3.4 Структура проекта и каркас MVC с активной моделью

Проект разделён на следующие компоненты:

- LifeModel – модель, содержащая дату рождения, расчёты и PropertyChangeSupport;
- MainView – главное окно, отображающее результат и PropertyChangeListener;
- LifeStatistics – класс статических коэффициентов;
- InputDialog – модальное окно ввода и проверки корректности даты;
- LifeController – контроллер, открывающий окно и передающий корректную дату модели;
- Main – точка входа, создающая компоненты MVC и запускающая интерфейс.



Рисунок 3.3 – Диаграмма типов и интерфейсов приложения

3.5 Модель

`LifeModel` хранит `birthDate` и выполняет расчёты. Метод `setBirthDate()` сохраняет новую дату и генерирует события через `PropertyChangeSupport`. Метод `getDaysLived()` использует `ChronoUnit.DAYS.between()`, `getAge()` – `Period.between()`. Остальные методы используют число прожитых дней и константы `LifeStatistics`.

3.6 Представление

`MainView` содержит заголовок, карточку результатов и кнопку «Ввести данные». Представление подписывается на модель. Метод `propertyChange()` вызывается автоматически и обновляет все значения. `InputDialog` содержит `JTextField`, кнопки «Сохранить» и «Отмена», подсказку формата и расширенную валидацию.

3.7 Контроллер

LifeController получает действие пользователя от MainView, открывает InputDialog, передаёт в него предыдущую дату и после подтверждения вызывает `model.setBirthDate()`. Controller не содержит расчётов статистических показателей.

3.8 Запоминание данных и обработка ошибок

Последняя корректная дата хранится в LifeModel. При повторном открытии окна она отображается в поле. Ошибочные данные не передаются в модель. Проверяются пустое поле, длина, разделители, символы, день, месяц, год, календарная корректность, будущее и максимальный возраст 150 лет.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В IntelliJ IDEA создано Java-приложение с графическим интерфейсом. Программа принимает дату рождения, рассчитывает статистические показатели, восстанавливает последнее корректное значение и сообщает об ошибочном вводе. Архитектура соответствует MVC с активной моделью.

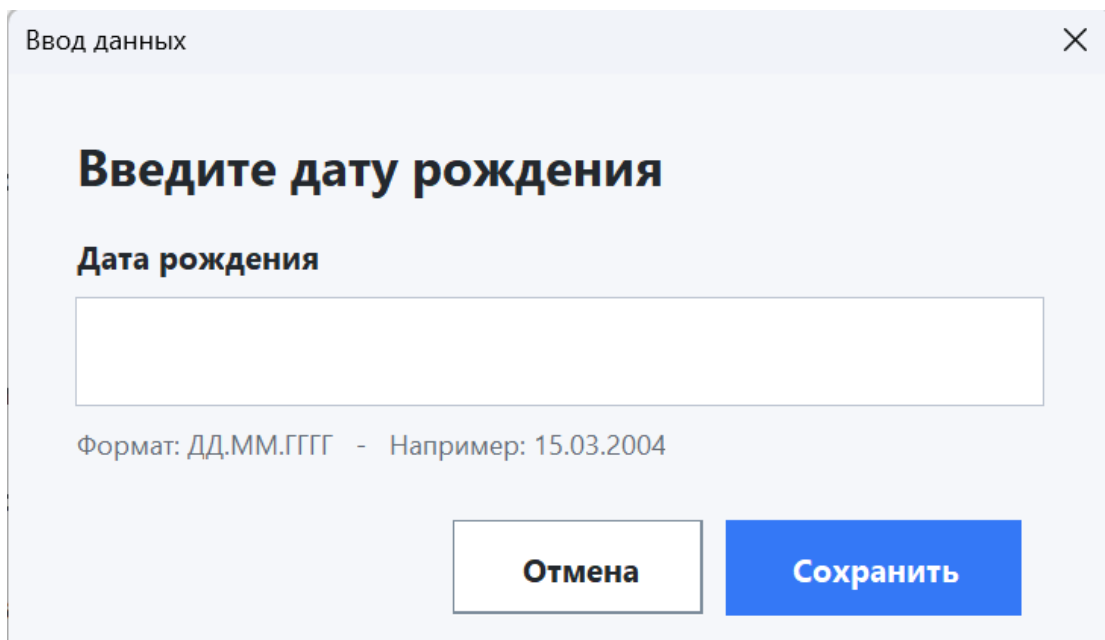
Калькулятор жизни

Статистическая оценка основных показателей жизни

Дата рождения	Не введена
Возраст	—
Прожито дней	—
Проспано	—
Морганий	—
Ударов сердца	—
Перекачано крови	—
Выпито воды	—
Смеха	—

Ввести данные

Рисунок 4.1 – Главная форма после запуска



Ввод данных

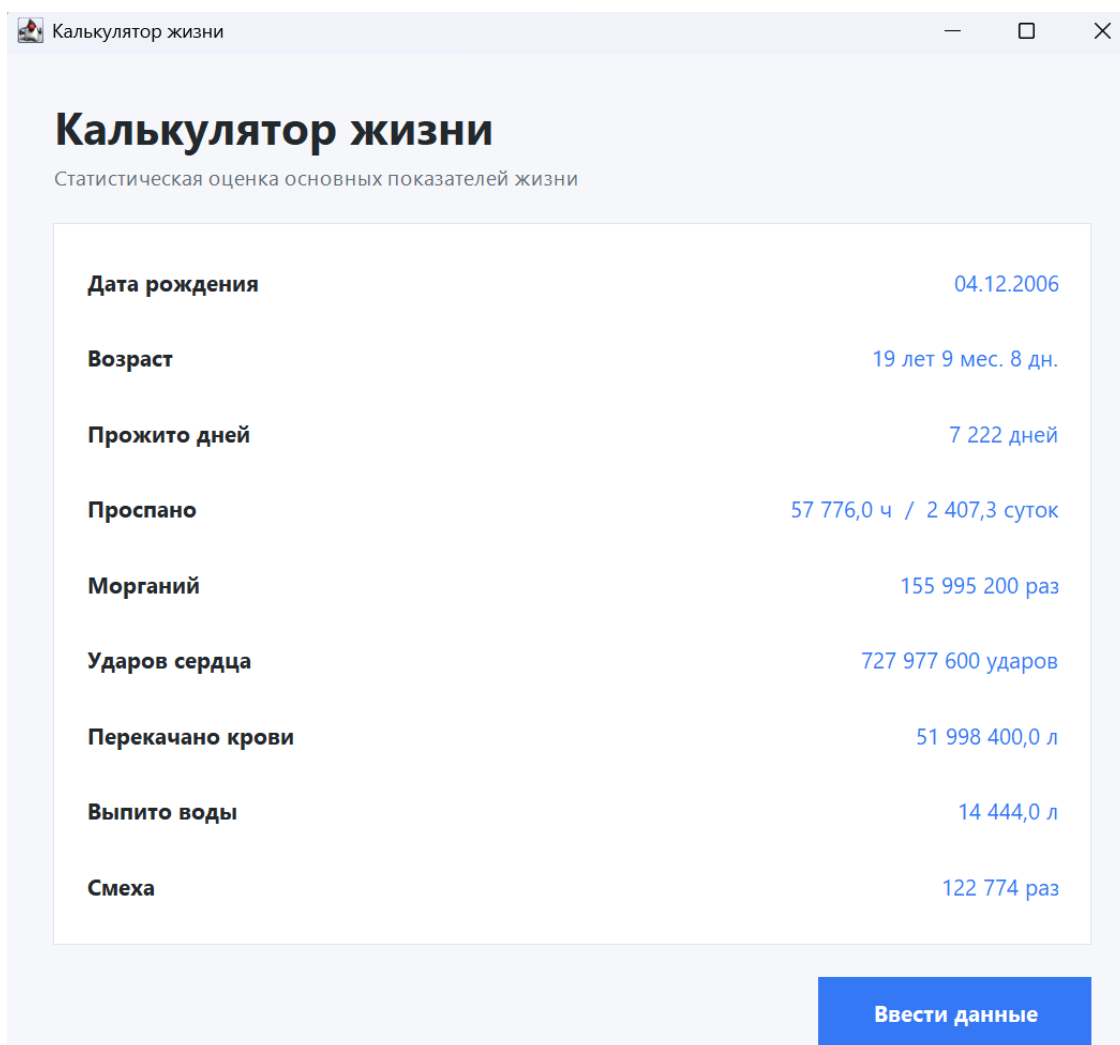
Введите дату рождения

Дата рождения

Формат: ДД.ММ.ГГГГ - Например: 15.03.2004

Отмена Сохранить

Рисунок 4.2 – Окно ввода данных



Калькулятор жизни

Статистическая оценка основных показателей жизни

Дата рождения	04.12.2006
Возраст	19 лет 9 мес. 8 дн.
Прожито дней	7 222 дней
Проспано	57 776,0 ч / 2 407,3 суток
Морганий	155 995 200 раз
Ударов сердца	727 977 600 ударов
Перекачано крови	51 998 400,0 л
Выпито воды	14 444,0 л
Смеха	122 774 раз

Ввести данные

Рисунок 4.3 – Результат вычисления

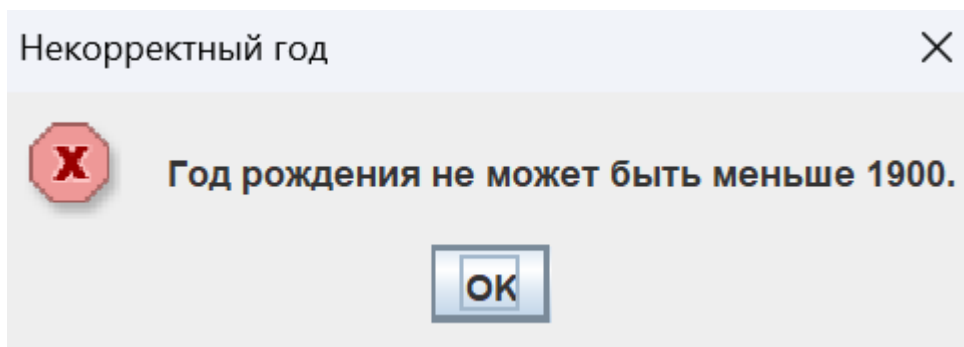


Рисунок 4.4 – Сообщение об ошибке

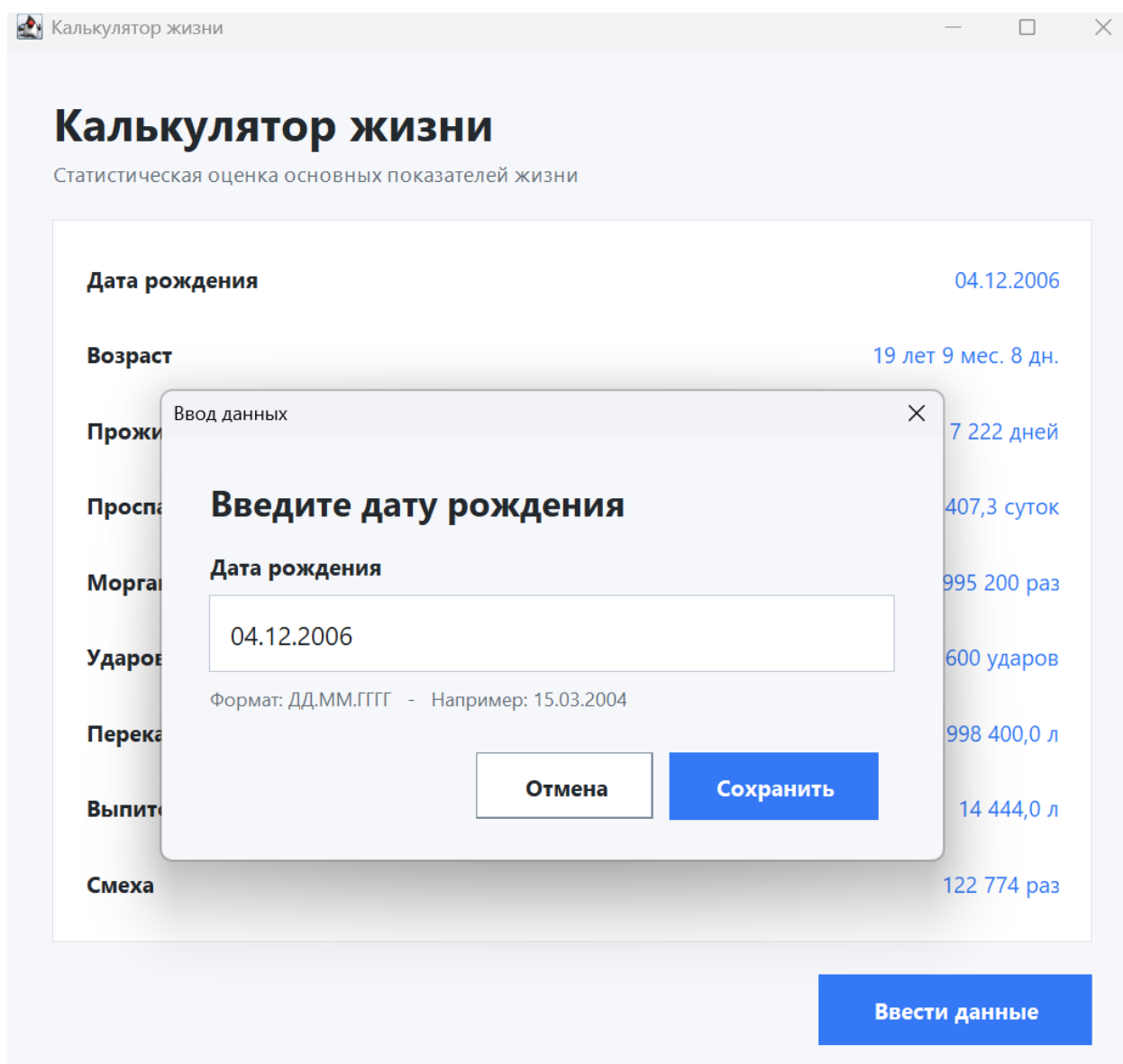


Рисунок 4.5 – Восстановленные данные при повторном вводе

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Какие компоненты входят в состав IDE?

В состав IDE входят редактор кода, система сборки и запуска, средства навигации и рефакторинга, отладчик, терминал, интеграция с системой контроля версий и механизм расширений.

Вопрос 2. Каковы основные приёмы работы с редактором кода?

К основным приёмам относятся автодополнение, автоматический импорт, форматирование, быстрые исправления, переход к объявлению, поиск использований, переименование и анализ предупреждений.

Вопрос 3. Каким образом выполняется сборка и запуск проекта?

В IntelliJ IDEA Java-проект можно собрать командой Build Project. Для запуска класса с методом main используется Run. Для отладки применяется Debug, при этом программа может останавливаться в заданных точках останова.

Вопрос 4. Что такое MVC?

MVC – архитектурный паттерн, разделяющий приложение на модель (данные и логику), представление (интерфейс) и контроллер (обработку действий пользователя). В данной работе моделью является LifeModel, представлением – MainView и InputDialog, контроллером – LifeController.

Вопрос 5. Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление об изменениях, поэтому контроллер сам инициирует обновление интерфейса. Активная модель публикует события при изменении состояния. В данной работе LifeModel использует PropertyChangeSupport, а MainView получает PropertyChangeEvent и автоматически обновляет метки.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные возможности IntelliJ IDEA при разработке Java-приложения с графическим интерфейсом, средства редактирования, сборки, запуска и отладки, а также архитектурный паттерн MVC.

Разработана программа «Калькулятор жизни». Реализован ручной ввод даты рождения в отдельном модальном окне без стандартного элемента выбора даты, многоуровневая проверка, сохранение и восстановление последнего корректного значения и вывод статистических результатов.

Расчёты выполняются по количеству прожитых полных дней и усреднённым коэффициентам. Полученные значения являются учебными оценками и не являются индивидуальными медицинскими измерениями.

Приложение построено по MVC с активной моделью. LifeModel уведомляет MainView посредством PropertyChangeSupport, благодаря чему представление автоматически обновляется после изменения состояния модели.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг исходного кода

Main.java

```
import controller.LifeController;
import model.LifeModel;
import view.MainView;
import javax.swing.*;
public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            LifeModel model = new LifeModel();
            MainView view = new MainView(model);
            new LifeController(model, view);
            view.setVisible(true);
        });
    }
}
```

LifeModel.java

```
package model;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.time.LocalDate;
import java.time.Period;
import java.time.temporal.ChronoUnit;

public class LifeModel {
    private LocalDate birthDate;

    private final PropertyChangeSupport propertyChangeSupport;
    public LifeModel() {
        propertyChangeSupport = new PropertyChangeSupport(this);
    }

    public void setBirthDate(LocalDate birthDate) {

        LocalDate oldDate = this.birthDate;
        this.birthDate = birthDate;
        propertyChangeSupport.firePropertyChange(
            "birthDate",
            oldDate,
            birthDate
        );

        propertyChangeSupport.firePropertyChange(
            "statistics",
            null,
            null
        );
    }

    public LocalDate getBirthDate() {
        return birthDate;
    }
}
```

```

    }

    public void addPropertyChangeListener(
        PropertyChangeListener listener
    ) {
        propertyChangeSupport.addPropertyChangeListener(listener);
    }

    public void removePropertyChangeListener(
        PropertyChangeListener listener
    ) {
        propertyChangeSupport.removePropertyChangeListener(listener);
    }

    private void checkBirthDate() {

        if (birthDate == null) {
            throw new IllegalStateException(
                "Дата рождения ещё не введена."
            );
        }
    }

    public long getDaysLived() {
        checkBirthDate();
        return ChronoUnit.DAYS.between(birthDate, LocalDate.now());
    }

    public Period getAge() {
        checkBirthDate();
        return Period.between(birthDate, LocalDate.now());
    }

    public double getSleepHours() {

        return getDaysLived()
            * LifeStatistics.SLEEP_HOURS_PER_DAY;
    }

    public double getSleepDays() {
        return getSleepHours() / 24.0;
    }

    public long getBlinks() {
        double result =
            getDaysLived()
                * 24.0
                * 60.0
                * LifeStatistics.BLINKS_PER_MINUTE;

        return Math.round(result);
    }

    public long getHeartBeats() {
        double result =
            getDaysLived()

```

```

        * 24.0
        * 60.0
        * LifeStatistics.HEART_BEATS_PER_MINUTE;

    return Math.round(result);
}

public double getBloodLiters() {
    return getDaysLived()
        * 24.0
        * 60.0
        * LifeStatistics.BLOOD_LITERS_PER_MINUTE;
}

public double getWaterLiters() {
    return getDaysLived()
        * LifeStatistics.WATER_LITERS_PER_DAY;
}

public long getLaughs() {
    double result =
        getDaysLived()
            * LifeStatistics.LAUGHS_PER_DAY;
    return Math.round(result);
}
}

```

LifeStatistics.java

```
package model;
```

```

public final class LifeStatistics {
    public static final double SLEEP_HOURS_PER_DAY = 8.0;
    public static final double BLINKS_PER_MINUTE = 15.0;
    public static final double HEART_BEATS_PER_MINUTE = 70.0;
    public static final double BLOOD_LITERS_PER_MINUTE = 5.0;
    public static final double WATER_LITERS_PER_DAY = 2.0;
    public static final double LAUGHS_PER_DAY = 17.0;

    private LifeStatistics() {
    }
}

```

InputDialog.java

```
package view;
```

```

import javax.swing.*.*;
import javax.swing.border.EmptyBorder;
import javax.swing.text.AbstractDocument;
import javax.swing.text.AttributeSet;
import javax.swing.text.BadLocationException;
import javax.swing.text.DocumentFilter;
import java.awt.*.*;
import java.awt.event.KeyEvent;
import java.time.DateTimeException;
import java.time.LocalDate;
import java.time.Month;
import java.time.Year;

```

```

public class InputDialog extends JDialog {

    private final JTextField dateField;
    private final JButton saveButton;
    private final JButton cancelButton;

    private LocalDate result;

    private static final Color BACKGROUND_COLOR =
        new Color(245, 247, 250);

    private static final Color PRIMARY_COLOR =
        new Color(52, 120, 246);

    private static final Color TEXT_COLOR =
        new Color(35, 40, 45);

    private static final Color SECONDARY_TEXT_COLOR =
        new Color(100, 108, 118);

    public InputDialog(JFrame parent, LocalDate previousDate) {

        super(parent, "Ввод данных", true);

        setSize(500, 300);

        setLocationRelativeTo(parent);

        setResizable(false);

        JPanel mainPanel = new JPanel(
            new BorderLayout(15, 15));

        mainPanel.setBackground(BACKGROUND_COLOR);

        mainPanel.setBorder(
            new EmptyBorder(25, 30, 25, 30)
        );

        JPanel headerPanel = new JPanel(new BorderLayout());

        headerPanel.setOpaque(false);

        JLabel titleLabel = new JLabel("Введите дату рождения");

        titleLabel.setFont(
            new Font("Segoe UI", Font.BOLD, 22)
        );

        titleLabel.setForeground(TEXT_COLOR);

        headerPanel.add(titleLabel, BorderLayout.WEST);

        mainPanel.add(headerPanel, BorderLayout.NORTH);

        JPanel centerPanel = new JPanel();

        centerPanel.setLayout(
            new BoxLayout(centerPanel, BoxLayout.Y_AXIS)
        );

        centerPanel.setOpaque(false);

        JLabel label = new JLabel("Дата рождения");
    }
}

```

```

label.setFont(
    new Font("Segoe UI", Font.BOLD, 14)
);

label.setForeground(TEXT_COLOR);

label.setAlignmentX(Component.LEFT_ALIGNMENT);

centerPanel.add(label);

centerPanel.add(Box.createVerticalStrut(8));

dateField = new JTextField();

dateField.setFont(
    new Font("Segoe UI", Font.PLAIN, 16)
);

dateField.setForeground(TEXT_COLOR);

dateField.setBackground(Color.WHITE);

dateField.setCaretColor(PRIMARY_COLOR);

dateField.setBorder(
    BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(
            new Color(190, 198, 210),
            1
        ),
        new EmptyBorder(10, 12, 10, 12)
    )
);

dateField.setMaximumSize(
    new Dimension(Integer.MAX_VALUE, 48)
);

dateField.setAlignmentX(Component.LEFT_ALIGNMENT);

((AbstractDocument) dateField.getDocument())
    .setDocumentFilter(new DateDocumentFilter()
);

if (previousDate != null) {
    dateField.setText(
        String.format(
            "%02d.%02d.%04d",
            previousDate.getDayOfMonth(),
            previousDate.getMonthValue(),
            previousDate.getYear()
        )
    );
}

centerPanel.add(dateField);

centerPanel.add(Box.createVerticalStrut(8));

JLabel hintLabel = new JLabel(
    "Формат: ДД.ММ.ГГГГ - Например: 15.03.2004"
);

```

```

hintLabel.setFont(
    new Font("Segoe UI", Font.PLAIN, 12)
);

hintLabel.setForeground(SECONDARY_TEXT_COLOR);

hintLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

centerPanel.add(hintLabel);

mainPanel.add(centerPanel, BorderLayout.CENTER);

JPanel buttonPanel = new JPanel(
    new FlowLayout(FlowLayout.RIGHT, 10, 0)
);

buttonPanel.setOpaque(false);

cancelButton = createSecondaryButton("Отмена");

saveButton = createPrimaryButton("Сохранить");

buttonPanel.add(cancelButton);
buttonPanel.add(saveButton);

mainPanel.add(buttonPanel, BorderLayout.SOUTH);

setContentPane(mainPanel);

saveButton.addActionListener(e -> saveDate());

cancelButton.addActionListener(e -> dispose());

getRootPane().setDefaultButton(saveButton);

getRootPane().registerKeyboardAction(
    e -> dispose(),
    KeyStroke.getKeyStroke(
        KeyEvent.VK_ESCAPE,
        0
    ),
    JComponent.WHEN_IN_FOCUSED_WINDOW
);

SwingUtilities.invokeLater(() -> {

    dateField.requestFocusInWindow();

    dateField.setCaretPosition(
        dateField.getText().length()
    );
});
}

private JButton createPrimaryButton(
    String text
) {

    JButton button = new JButton(text);

    button.setFont(
        new Font("Segoe UI", Font.BOLD, 14)
    );
}

```

```

        button.setForeground(Color.WHITE);

        button.setBackground(PRIMARY_COLOR);

        button.setFocusPainted(false);

        button.setBorderPainted(false);

        button.setOpaque(true);

        button.setPreferredSize(
            new Dimension(130, 42)
        );

        return button;
    }

    private JButton createSecondaryButton(
        String text
    ) {

        JButton button = new JButton(text);

        button.setFont(
            new Font("Segoe UI", Font.BOLD, 14)
        );

        button.setForeground(TEXT_COLOR);

        button.setBackground(Color.WHITE);

        button.setFocusPainted(false);

        button.setPreferredSize(
            new Dimension(110, 42)
        );

        return button;
    }

    private void saveDate() {

        String text =
            dateField.getText().trim();

        if (text.isEmpty()) {

            showError(
                "Дата не введена",
                "Введите дату рождения в формате ДД.ММ.ГГГГ."
            );

            dateField.requestFocusInWindow();

            return;
        }

        if (text.length() != 10) {

            showError(
                "Неполная дата",
                "Дата должна содержать 10 символов.\n"
                + "Используйте формат ДД.ММ.ГГГГ."
            );
        }
    }

```

```

    );

    dateField.requestFocusInWindow();

    return;
}

if (text.charAt(2) != '.' || text.charAt(5) != '.') {

    showError(
        "Неверный формат",
        "В качестве разделителей используйте точки:\n"
        + "ДД.ММ.ГГГГ"
    );

    dateField.requestFocusInWindow();

    return;
}

for (int i = 0; i < text.length(); i++) {

    if (i == 2 || i == 5) {
        continue;
    }

    if (!Character.isDigit(
        text.charAt(i)
    )) {

        showError(
            "Недопустимый символ",
            "Дата должна содержать только цифры\n"
            + "и точки."
        );

        dateField.requestFocusInWindow();

        return;
    }
}

try {

    int day = Integer.parseInt(
        text.substring(0, 2)
    );

    int month = Integer.parseInt(
        text.substring(3, 5)
    );

    int year = Integer.parseInt(
        text.substring(6, 10)
    );

    if (day < 1 || day > 31) {

        showError(
            "Некорректный день",
            "День должен находиться в диапазоне от 01 до 31."
        );

        dateField.requestFocusInWindow();
    }
}

```

```

        return;
    }

    if (month < 1 || month > 12) {

        showError(
            "Некорректный месяц",
            "Месяц должен находиться в диапазоне от 01 до 12."
        );

        dateField.requestFocusInWindow();

        return;
    }

    int currentYear =
        LocalDate.now().getYear();

    if (year < 1900) {

        showError(
            "Некорректный год",
            "Год рождения не может быть меньше 1900."
        );

        dateField.requestFocusInWindow();

        return;
    }

    if (year > currentYear) {

        showError(
            "Некорректный год",
            "Год рождения не может быть больше "
                + currentYear
                + "."
        );

        dateField.requestFocusInWindow();

        return;
    }

    Month selectedMonth = Month.of(month);

    int maxDays =
        selectedMonth.length(Year.isLeap(year));

    if (day > maxDays) {

        String monthName = getMonthName(month);

        showError(
            "Несуществующая дата",
            "В месяце "
                + monthName
                + " "
                + year
                + " года нет "
                + day
                + "-го дня."
        );
    }

```

```

        dateField.requestFocusInWindow();

        return;
    }

    LocalDate date =
        LocalDate.of(year, month, day);

    LocalDate today = LocalDate.now();

    if (date.isAfter(today)) {

        showError(
            "Дата находится в будущем",
            "Дата рождения не может быть позже сегодняшнего дня."
        );

        dateField.requestFocusInWindow();

        return;
    }

    LocalDate minimumDate =
        today.minusYears(150);

    if (date.isBefore(minimumDate)) {

        showError(
            "Некорректная дата",
            "Возраст человека не должен превышать 150 лет."
        );

        dateField.requestFocusInWindow();

        return;
    }

    result = date;

    dispose();

} catch (DateTimeException |
        NumberFormatException exception) {

    showError(
        "Ошибка даты",
        "Не удалось распознать введённую дату.\n"
        + "Проверьте правильность данных."
    );

    dateField.requestFocusInWindow();
}

}

private void showError(String title, String message
) {

    JOptionPane.showMessageDialog(
        this,
        message,
        title,
        JOptionPane.ERROR_MESSAGE
    );
}

```

```

    }

    private String getMonthName(int month) {

        String[] months = {
            "январе",
            "феврале",
            "марте",
            "апреле",
            "мае",
            "июне",
            "июле",
            "августе",
            "сентябре",
            "октябре",
            "ноябре",
            "декабре"
        };

        return months[month - 1];
    }

    public LocalDate getResult() {
        return result;
    }

    private static class DateDocumentFilter
        extends DocumentFilter {

        @Override
        public void insertString(
            FilterBypass fb,
            int offset,
            String string,
            AttributeSet attr
        ) throws BadLocationException {

            if (string == null) {
                return;
            }

            StringBuilder filtered =
                new StringBuilder();

            for (char c : string.toCharArray()) {

                if (Character.isDigit(c)
                    || c == '.') {

                    filtered.append(c);
                }
            }

            String current =
                fb.getDocument().getText(
                    0,
                    fb.getDocument().getLength()
                );

            if (current.length()
                + filtered.length() <= 10) {

                fb.insertString(
                    offset,

```

```

        filtered.toString(),
        attr
    );
    }
}

@Override
public void replace(
    FilterBypass fb,
    int offset,
    int length,
    String text,
    AttributeSet attrs
) throws BadLocationException {

    if (text == null) {
        text = "";
    }

    StringBuilder filtered =
        new StringBuilder();

    for (char c : text.toCharArray()) {

        if (Character.isDigit(c)
            || c == '.') {

            filtered.append(c);
        }
    }

    String current =
        fb.getDocument().getText(
            0,
            fb.getDocument().getLength()
        );

    int newLength =
        current.length()
        - length
        + filtered.length();

    if (newLength <= 10) {

        fb.replace(
            offset,
            length,
            filtered.toString(),
            attrs
        );
    }
}
}
}

```

MainView.java

```

package view;

import model.LifeModel;

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;

```

```

import java.beans.PropertyChangeEvent;
import java.beans.PropertyChangeListener;
import java.time.Period;

public class MainView extends JFrame
    implements PropertyChangeListener {

    private final JButton inputButton;

    private final JLabel birthDateLabel;
    private final JLabel ageLabel;
    private final JLabel daysLabel;
    private final JLabel sleepLabel;
    private final JLabel blinksLabel;
    private final JLabel heartLabel;
    private final JLabel bloodLabel;
    private final JLabel waterLabel;
    private final JLabel laughsLabel;

    private static final Color BACKGROUND =
        new Color(245, 247, 250);

    private static final Color CARD_BACKGROUND =
        Color.WHITE;

    private static final Color PRIMARY =
        new Color(52, 120, 246);

    private static final Color TEXT =
        new Color(35, 40, 45);

    private static final Color SECONDARY_TEXT =
        new Color(100, 108, 118);

    private static final Color BORDER =
        new Color(225, 229, 235);

    public MainView(LifeModel model) {

        setTitle("Калькулятор жизни");

        setSize(720, 680);

        setMinimumSize(new Dimension(650, 600));

        setLocationRelativeTo(null);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JPanel mainPanel = new JPanel(
            new BorderLayout(20, 20));

        mainPanel.setBackground(BACKGROUND);

        mainPanel.setBorder(
            new EmptyBorder(25, 30, 25, 30)
        );

        JPanel headerPanel =
            new JPanel(new BorderLayout());

        headerPanel.setOpaque(false);

        JLabel titleLabel =

```

```

        new JLabel("Калькулятор жизни");

titleLabel.setFont(
    new Font("Segoe UI", Font.BOLD, 28)
);

titleLabel.setForeground(TEXT);

JLabel subtitleLabel =
    new JLabel("Статистическая оценка основных показателей
жизни");

subtitleLabel.setFont(
    new Font("Segoe UI", Font.PLAIN, 13)
);

subtitleLabel.setForeground(SECONDARY_TEXT);

JPanel titlePanel = new JPanel();

titlePanel.setLayout(
    new BoxLayout(titlePanel, BoxLayout.Y_AXIS)
);

titlePanel.setOpaque(false);

titlePanel.add(titleLabel);

titlePanel.add(Box.createVerticalStrut(4));

titlePanel.add(subtitleLabel);

headerPanel.add(titlePanel, BorderLayout.WEST);

mainPanel.add(headerPanel, BorderLayout.NORTH);

JPanel cardPanel =
    new JPanel(new GridLayout(0, 2, 0, 0)
);

cardPanel.setBackground(CARD_BACKGROUND);

cardPanel.setBorder(
    BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(BORDER),
        new EmptyBorder(10, 20, 10, 20)
    )
);

birthDateLabel = createValueLabel("Не введена");

ageLabel = createValueLabel("-");

daysLabel = createValueLabel("-");

sleepLabel = createValueLabel("-");

blinksLabel = createValueLabel("-");

heartLabel = createValueLabel("-");

bloodLabel = createValueLabel("-");

waterLabel = createValueLabel("-");

```

```

        laughsLabel = createValueLabel("-");

        addRow(cardPanel, "Дата рождения", birthDateLabel);

        addRow(cardPanel, "Возраст", ageLabel);

        addRow(cardPanel, "Прожито дней", daysLabel);

        addRow(cardPanel, "Проспано", sleepLabel);

        addRow(cardPanel, "Морганий", blinksLabel);

        addRow(cardPanel, "Ударов сердца", heartLabel);

        addRow(cardPanel, "Перекачано крови", bloodLabel);

        addRow(cardPanel, "Выпито воды", waterLabel);

        addRow(cardPanel, "Смеха", laughsLabel);

        mainPanel.add(cardPanel, BorderLayout.CENTER);

        JPanel bottomPanel =
            new JPanel(new BorderLayout());

        bottomPanel.setOpaque(false);

        inputButton = new JButton("Ввести данные"
        );

        inputButton.setFont(
            new Font("Segoe UI", Font.BOLD, 14)
        );

        inputButton.setForeground(Color.WHITE);

        inputButton.setBackground(PRIMARY);

        inputButton.setFocusPainted(false);

        inputButton.setBorderPainted(false);

        inputButton.setOpaque(true);

        inputButton.setPreferredSize(
            new Dimension(170, 44)
        );

        bottomPanel.add(inputButton, BorderLayout.EAST);

        mainPanel.add(bottomPanel, BorderLayout.SOUTH);

        setContentPane(mainPanel);

        model.addPropertyChangeListener(this);
    }

    private JLabel createValueLabel(
        String text
    ) {

        JLabel label = new JLabel(text);

```

```

        label.setFont(
            new Font("Segoe UI", Font.PLAIN, 14)
        );

        label.setForeground(PRIMARY);

        label.setHorizontalAlignment(SwingConstants.RIGHT);

        return label;
    }

    private void addRow(
        JPanel panel,
        String name,
        JLabel value
    ) {

        JLabel nameLabel = new JLabel(name);

        nameLabel.setFont(
            new Font("Segoe UI", Font.BOLD, 14)
        );

        nameLabel.setForeground(TEXT);

        nameLabel.setBorder(
            new EmptyBorder(12, 0, 12, 0)
        );

        value.setBorder(
            new EmptyBorder(12, 0, 12, 0)
        );

        panel.add(nameLabel);
        panel.add(value);
    }

    public JButton getInputButton() {
        return inputButton;
    }

    @Override
    public void propertyChange(
        PropertyChangeEvent event
    ) {

        if (!event.getPropertyName().equals("birthDate")
            && !event.getPropertyName().equals("statistics")) {

            return;
        }

        LifeModel model =
            (LifeModel) event.getSource();

        if (model.getBirthDate() == null) {
            return;
        }

        birthDateLabel.setText(
            String.format(
                "%02d.%02d.%04d",
                model.getBirthDate().getDayOfMonth(),

```

```

        model.getBirthDate().getMonthValue(),
        model.getBirthDate().getYear()
    );

    Period age = model.getAge();

    ageLabel.setText(
        age.getYears()
        + " лет "
        + age.getMonths()
        + " мес. "
        + age.getDays()
        + " дн."
    );

    daysLabel.setText(
        String.format(
            "%,d дней",
            model.getDaysLived()
        )
    );

    sleepLabel.setText(
        String.format(
            "%,.1f ч / %,.1f суток",
            model.getSleepHours(),
            model.getSleepDays()
        )
    );

    blinksLabel.setText(
        String.format(
            "%,d раз",
            model.getBlinks()
        )
    );

    heartLabel.setText(
        String.format(
            "%,d ударов",
            model.getHeartBeats()
        )
    );

    bloodLabel.setText(
        String.format(
            "%,.1f л",
            model.getBloodLiters()
        )
    );

    waterLabel.setText(
        String.format(
            "%,.1f л",
            model.getWaterLiters()
        )
    );

    laughsLabel.setText(
        String.format(
            "%,d раз",
            model.getLaughs()
        )
    );

```

```

        );
    }
}

```

LifeController.java

```

package controller;

import model.LifeModel;
import view.InputDialog;
import view.MainView;

import java.time.LocalDate;

public class LifeController {

    private final LifeModel model;
    private final MainView view;

    public LifeController(
        LifeModel model,
        MainView view
    ) {

        this.model = model;

        this.view = view;

        view.getInputButton()
            .addActionListener(e -> openInputDialog()
                );
    }

    private void openInputDialog() {

        LocalDate previousDate = model.getBirthDate();

        InputDialog dialog = new InputDialog(view, previousDate);

        dialog.setVisible(true);

        LocalDate newDate = dialog.getResult();

        if (newDate != null) {

            model.setBirthDate(newDate);
        }
    }
}

```